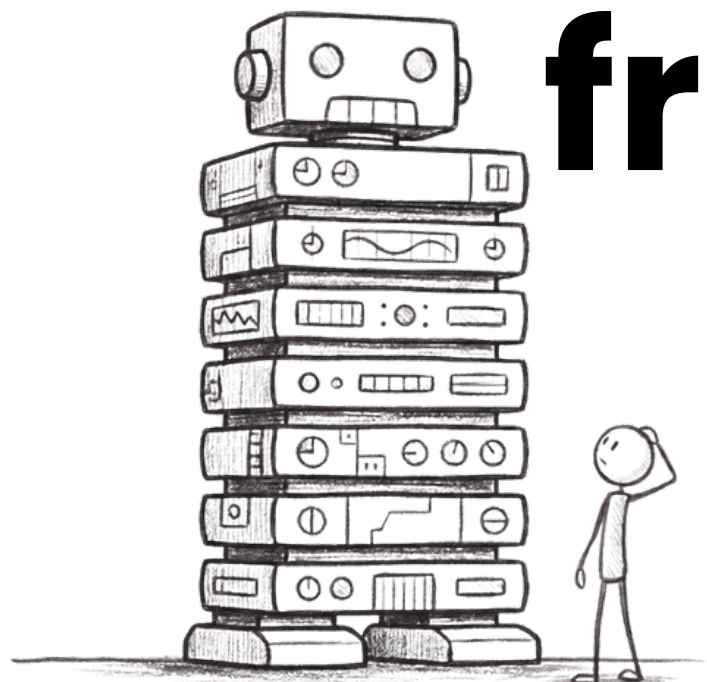


BUILD **Production** **AI AGENTS** **from scratch** **in 2026**

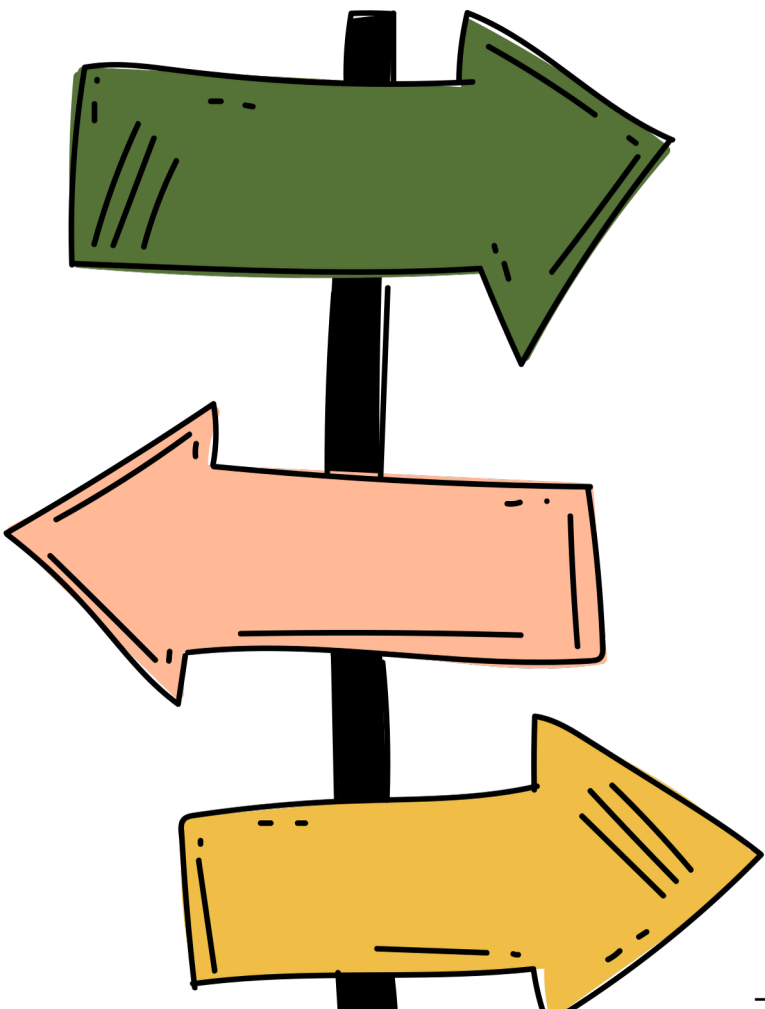


Dr. Maryam Miradi
Chief AI Scientist

Built from 20+ years in AI.
Field-tested. Zero fluff.

“Over the past year, I’ve built over 400+ AI agents—for my training programs and real-world deployments across multiple industries. With 20+ years of hands-on AI experience, I’ve seen exactly what separates agents that just run from those that deliver real results. This guide gives you the production stack I wish I had when I started. Every framework decision, every architecture pattern, and every failure mode I’ve encountered across 12 industries, condensed into one resource you can use today.”

–Dr. Maryam Miradi



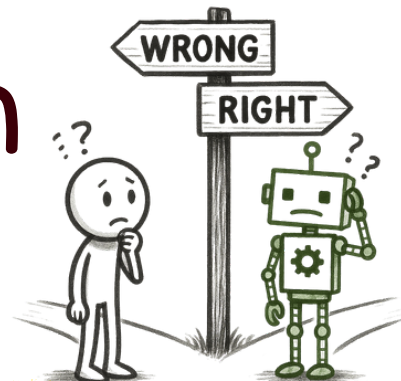


MARYAM MIRADI



Dr. Maryam Miradi

How to Build AI AGENTS from Scratch



Step 1 » DEFINE THE AGENT'S ROLE AND GOAL

- * What will your agent do?
- * Who is it helping?
- * What kind of output will it generate?
- **Example: A medical assistant agent that reads X-rays, summarizes findings, and speaks results.**

Step 2 » DESIGN STRUCTURED INPUT & OUTPUT

- * Use Pydantic AI or JSON Schemas to define what the agent receives and returns.
- * Avoid messy text — think like an API.
- **Tool: Pydantic AI, LangChain Output Parsers**

Step 3 » TUNE BEHAVIOR & ADD PROTOCOL

- * Start with role-based system prompts
- * Use Prompt Tuning or Prefix Tuning
- * Use MCP to standardize how your agents
- **Tools: MCP, GPT-4, Claude, Prompt Tuning**

Step 4 » ADD REASONING AND TOOL USE

- * Equip the agent with reasoning frameworks:
 - ☆ ReAct (Reasoning + Action)
 - ☆ Chain-of-Thought
- * Allow access to tools like web search, code interpreters, or document retrievers.
- **Tools: LangChain, OpenAI Tools, ReAct Framework**

Step 5 » STRUCTURE MULTI-AGENT LOGIC (IF NEEDED)

- * Use orchestration frameworks to define agent roles and coordination.
- * Create Planner, Researcher, Reporter agents — each with its own input/output schema.
- **Tools: CrewAI, LangGraph, OpenAI Swarm**

Step 6 » ADD MEMORY AND LONG-TERM CONTEXT (RAG)

- * Does your agent need to remember what happened earlier?
- * Use conversational memory, summary memory, or vector-based memory.
- **Tools: Zep, LangChain Memory, ChromaDB, FAISS**

Step 7 » ADD VOICE OR VISION CAPABILITIES (OPTIONAL)

- * Text-to-speech: Use Coqui or ElevenLabs
- * Image understanding: Use GPT-4o or LLaMA 3.2 Vision
- **Let your agent see and speak.**

Step 8 » DELIVER THE OUTPUT

- * Format outputs into Markdown → PDF or JSON
- * Output must be readable and parsable
- **Tools: Pydantic AI, LangChain Parsers**

Step 9 » WRAP IN A UI

- * Create a front-end
- * Use Gradio, Streamlit, or FastAPI
- **This is what turns your agent into a product.**

Step 10 » EVALUATE AND MONITOR

- * Run test prompts and toolchains to check reliability.
- * Use logs, benchmarks, and feedback to improve over time.
- **Tools: MCP Logs, OpenAI Evaluation API, Custom Metrics Dashboards**

www.maryammiradi.com

COMPLETE TAXONOMY OF AI Agents Memory



Dr. Maryam Miradi



BY FUNCTION

- 1) Sensory Memory: Raw input buffer that temporarily holds what the agent has just perceived before processing.
- 2) Working Memory: Active reasoning scratch pad where the agent performs step-by-step thinking during a reasoning cycle.
- 3) Episodic Memory: Storage of specific past interactions and events that enables personalization and continuity across sessions.
- 4) Semantic Memory: General domain knowledge and factual information typically stored in a retrieval or vector layer.
- 5) Procedural Memory: Encoded skills, workflows, rules, and behavioral instructions that guide how the agent acts.
- 6) Strategic / Meta Memory: Memory of past decisions and outcomes that allows the agent to reflect, evaluate performance, and improve over time.

BY SCOPE

- 1) Private Agent Memory: Memory accessible to a single agent only, isolated from other agents to prevent conflicts.
- 2) Shared Agent Memory: Memory accessible by multiple agents simultaneously, enabling coordination but requiring synchronization to avoid conflicts.
- 3) User Memory: Persistent memory tied to an individual user, enabling personalization across sessions.
- 4) Global / World Memory: Shared system-wide memory accessible to all agents and users, forming the collective knowledge base.

BY DURATION

- 1) Short-Term / In-Context Memory: Lives inside the model's context window.
- 2) Long-Term / External Memory: Stored outside the model in persistent storage.

Common Tools:
LangChain ConversationBufferMemory
OpenAI message array, LangGraph AgentState,
PydanticAI RunContext,
ChromaDB, Qdrant, FAISS, Pinecone
PostgreSQL, MongoDB, Redis, Zep
Mem0

BY STORAGE TYPE

- 1) In-Context Memory: Raw text stored directly in the prompt window.
- 2) Vector / Semantic Memory: Embeddings stored in a vector database.
- 3) Structured / Episodic Memory: Stored in SQL or NoSQL databases.
- 4) Cache Memory: Key-value storage for repeated computations.
- 5) Graph Memory: Entities and relationships stored in a knowledge graph.

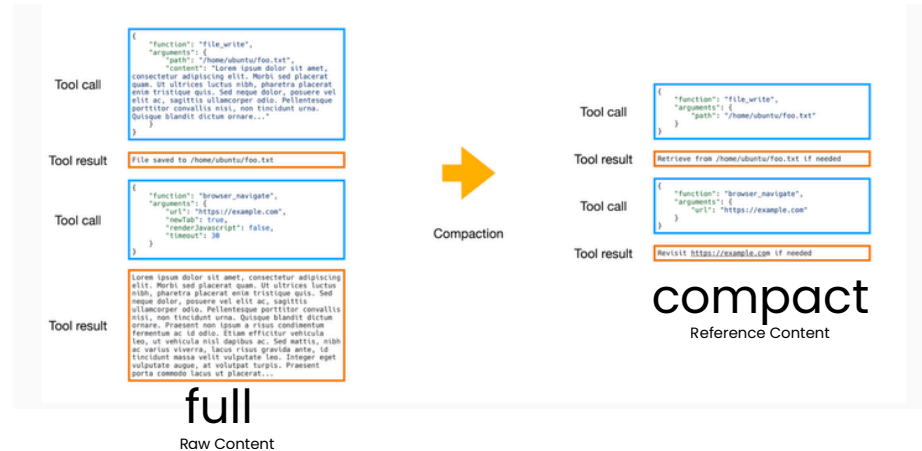
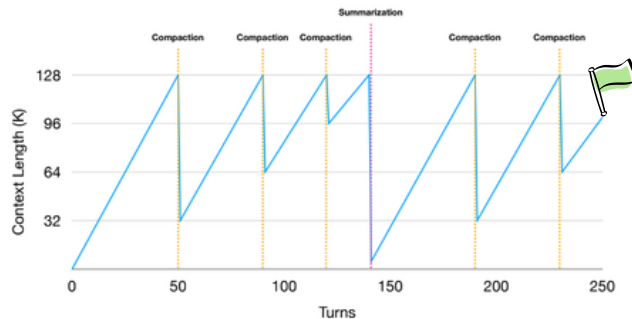
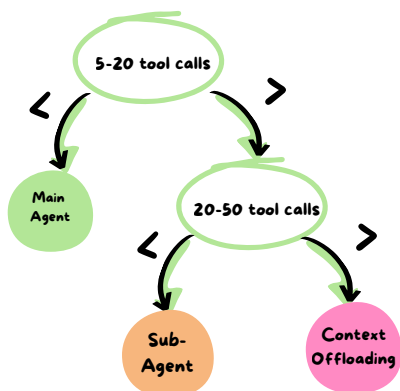
Common Tools:
LangChain ConversationBufferMemory, LangGraph state, OpenAI message array, PydanticAI RunContext, ChromaDB, FAISS, Qdrant, Pinecone, Weaviate, MongoDB Atlas Vector Search, LangChain VectorStoreRetriever, PostgreSQL, MongoDB, SQLite, LangChain SQLChatMessageHistory, Supabase, RedisUpstash, Redis, CPTCache, LangChain CacheBackedEmbeddings, In-memory Python dictionaries, Neo4j, LangChain Neo4j integration, Microsoft GraphRAG, FalkorDB, NebulaGraph, LlamaIndex Knowledge Graph Index

Common Tools: LangChain HumanMessage / AIMessage buffers, PydanticAI input validators, OpenAI message array, LangGraph AgentState, OpenAI Swarm context variables, PydanticAI RunContext, CrewAI task context, LangChain ConversationBufferMemory, Zep, Mem0, LangChain ConversationBufferWindowMemory, LangChain ConversationSummaryMemory, LangChain ConversationSummaryBufferMemory, Redis, PostgreSQL, ChromaDB, FAISS, Qdrant, Pinecone, Weaviate, LlamaIndex, LangChain VectorStoreRetriever, Docling, LlamaParse, Unstructured.io, System prompt, LangChain prompt templates, LangGraph node definitions and edge conditions, CrewAI task definitions, PydanticAI tool schemas, DSPy, LangGraph checkpointing, LangSmith, LangChain callbacks, Weights & Biases, PostgreSQL feedback loop tables, Reflexion pattern

Common Tools: LangGraph node-level state, PydanticAI RunContext, in-memory Python objects, LangGraph shared state, Redis with pub/sub, CrewAI shared memory, OpenAI Swarm context variables, PostgreSQL with row-level locking, Apache Kafka, Mem0, Zep, PostgreSQL user profile tables, MongoDB user documents, Supabase, ChromaDB, Qdrant, Neo4j, PostgreSQL, LlamaIndex, Microsoft GraphRAG

PRODUCTION MEMORY FAILURES NOBODY TALKS ABOUT

- Memory Poisoning
Bad data written to long-term memory corrupts future retrievals.
- Memory Drift
Agent behavior degrades as biased episodic data accumulates.
- Context Stuffing
Overloading the context window destroys reasoning quality.
- Retrieval Hallucination
Agent retrieves a chunk, misinterprets it, and fabricates details.
- Memory Conflicts
Multiple agents write contradictory facts simultaneously.
- Cache Staleness
Expired cached data leads to incorrect decisions.



HOW TO APPLY Context Engineering TO AIAgents



Dr. Maryam Miradi



Law #1
Offload Everything
Token-Heavy

Keep paths, drop contents.
File system is external memory.

Law #2
Compact First,
Summarize Last

When you must summarize:
Use structured schemas,
never free-form prompts.

Law #3
Set Alarms Early
(128K-200K
Tokens)

Monitor context size
continuously. When you hit
200K, automatically trigger
compaction or offloading.

Law #4
Split by Task Type,
Not Job Titles

✗ Designer Agent,
Developer Agent, Manager
Agent
✓ Research Agent
(gathers), Writer Agent
(synthesizes)

Law #5
3-Layer Tool
Design

Basic → Utilities → Code
Small interface,
infinite capabilities.

Law #6
Structured Beats
Free-Form

✗ "Summarize the
conversation"
✓ "Fill this schema:
- Files modified: [list]
- Key findings: [list]
- Next steps: [list]
- Open questions: [list]"

Law #7
Simpler Systems
Win

When in doubt, remove
complexity.

- What to remove:
- Unnecessary sub-agents
- Over-engineered workflows
- Excessive validation steps

Law #8
Monitor
Continuously

What to measure:

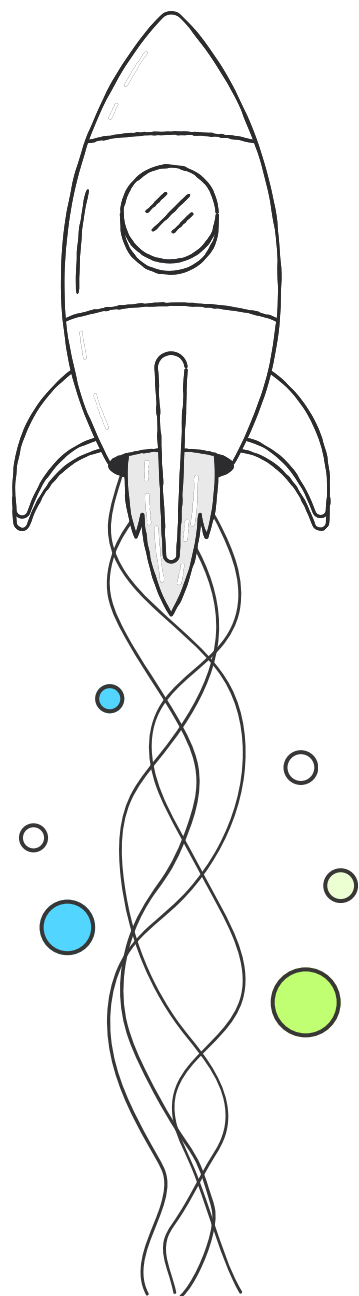
- Context size
- Retrieval frequency
- Sub-agent coordination failures
- Task completion rate



Dr. Maryam Miradi



HOW TO SURVIVE MEMORY POISONING & PROMPT INJECTION FOR AI AGENTS



BOUNDED EXECUTION

1

Constrain Tool Access Red agents without execution boundaries escape simulation environments

MEMORY ISOLATION

2

Separate Context Scopes
Poisoned memory persists across sessions and corrupts all future decisions

INPUT VALIDATION

3

Sanitize ALL Data Sources
Enrichment feeds and threat intel can inject malicious instructions

CONTEXT SEPARATION

4

Instructions $\neq$ Data Agents must distinguish between commands and external information

5

OUTPUT VALIDATION

Policy Gates Before Action
Validate every decision against security rules before execution

6

RAG QUALITY CONTROL

Version Controlled Patterns
Poisoned retrieval patterns corrupt reasoning in all future queries

7

AUDIT EVERYTHING

Immutable Decision Logs
Track every tool call, reasoning step, and memory update

8

SANDBOXED ENVIRONMENTS

Isolated Execution Spaces 75% of malicious commands succeed without proper sandboxing

+ HUMAN ESCALATION

HITL for High-Impact Actions Irreversible operations require explicit human approval gates

LLM Tuning Ladder 9-step

from Prompting to Full Fine-Tuning



Dr. Maryam Miradi



Train From Scratch

For: Full sovereignty
Use cases: National models
Tools: PyTorch, Megatron-LM

Continued Pretraining

For: domain corpus adaptation
Use cases: Biomedical, finance
Tools: Hugging Face Accelerate, DeepSpeed

Full Fine-Tuning

For: performance gaps, strict domain control
Use cases: Compliance regulated industries
Tools: OpenAI fine-tuning API, Hugging Face Trainer

LoRA / Adapters (PEFT)

For: Labeled data, performance improvement, low-cost
Use cases: Legal language, medical QA
Tools: LoRA, QLoRA, Hugging Face PEFT

Prefix Tuning

For: Stronger parameter-efficient adaptation
Use cases: Context biasing without full fine-tune
Tools: Hugging Face PEFT

Prompt Tuning (Soft Prompts)

For: Lightweight task specialization
Use cases: Domain adaptation, style biasing
Tools: Hugging Face PEFT

Structured Prompt Engineering

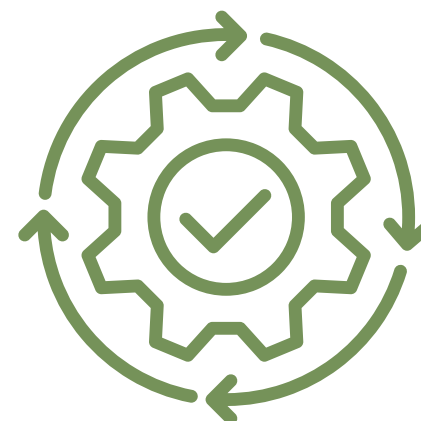
For: Reasoning control, guardrails, agents
Use cases: Reliable orchestration, schema enforcement
Tools: LangGraph, CrewAI, PydanticAI

Few-Shot Prompting

For: Output stability, formatting control
Use cases: Structured JSON, style alignment
Tools: OpenAI function calling, LangChain output parsers

Zero-Shot Prompting

For: Prototyping, simple tasks, no data
Use cases: Basic classification, extraction, brainstorming
Tools: OpenAI API, Anthropic Claude, Google Gemini



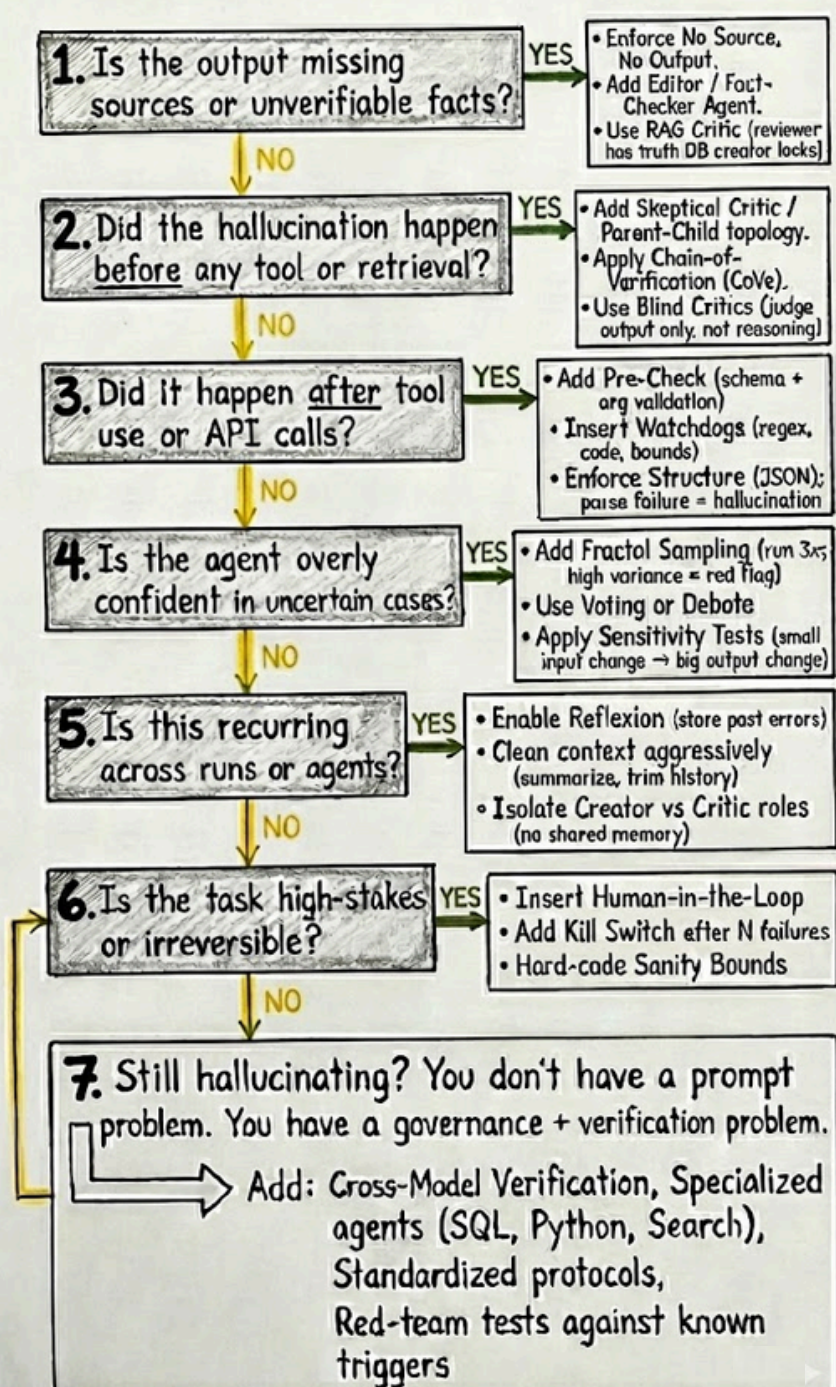


Dr. Maryam Miradi



How to Mitigate LLM Hallucination for AI Agents

LLM Hallucination Decision Tree



1. Agent hallucinations are "physically consequential"—unlike chatbots, agents can delete files or call wrong APIs.
2. The "Parent-Child" topology: One agent drafts, a second agent specifically critiques.
3. Separate the prompts: The "Critic" agent must not see the "Creator's" reasoning, only the output.
4. Multi-agent debate: Two agents arguing a point yields higher accuracy than one agent thinking twice.
5. Assign "Persona" roles: A "Skeptical Reviewer" persona detects 20% more errors than a generic prompt.
6. Cross-Model Verification: Use cheap models (Llama-3-8b) to draft, and SOTA models (GPT-4o) to verify.
7. Black-box Watchdogs: Use non-LLM scripts (code compilers, regex) to validate outputs before execution.
8. Probabilistic Fractal Sampling: Query the agent 3x. High variance in answers = Hallucination.
9. HalMit Framework: Define a "generalization bound"—if the agent steps outside known data, flag it.
10. Structure is safety: Enforce JSON output. If it doesn't parse, it's a structural hallucination.
11. Thought-Action-Observation loops (ReAct) prevent "blind" guessing.
12. Tool Argument Validation: Check generated API parameters against the schema before sending the request.
13. Reflexion: Agents must store past errors in memory to avoid repeating specific hallucinations.
14. The "Editor" Pattern: A dedicated agent tasked solely with removing unverified claims from the final text.
15. Citation-Based Verification: Agents must provide a URL/File ID for every claim. No source = No output.
16. Vertical Specialization: A SQL agent and a Python agent hallucinate less than one "Generalist" agent.
17. Human-in-the-Loop: For high-stakes actions (POST/DELETE requests), force a human approval step.
18. Negative Constraints: Explicitly prompt what the agent should not do (e.g., "Do not invent IDs").
19. Confidence Scoring: Ask the Reviewer agent to rate the answer 1-10. Discard anything < 8.
20. Consensus Voting (Self-Consistency): Run 3 agent instances; take the majority answer.
21. State Grounding: Agents must "Observe" the current state (e.g., list files) immediately before "Acting."
22. Fuzzy Logic Rules: Use fuzzy matching to validate text outputs where exact string matching fails.
23. Sanity Checks: Hard-coded logic bounds (e.g., "Refund amount cannot be > Original Price").
24. Context Cleaning: Long context windows increase hallucinations. Summarize history often.
25. Isolation: Don't let the Reviewer agent fix the error; have it send feedback to the Creator to fix it.
26. Prompt Sensitivity Testing: If changing a comma alters the decision, the agent is hallucinating.
27. Fact-Checking Tool: Give the agent a "Google Search" tool specifically to verify its own internal knowledge.
28. Loop Limits: If an agent retries a failed action 3x, kill the process to prevent spiraling.
29. Domain-Specific RAG: Ground the critic agent with a "Truth" vector database the creator agent cannot see.
30. Standardized Communication: Use protocols (like ACL) so agents don't misunderstand each other's messages.
31. Evaluation Sets: Continually test your agent swarm against known hallucination triggers, not just happy paths.



Dr. Maryam Miradi



CRITICAL 9 Checks OF PRODUCTION AI Agents

BEFORE → DURING → AFTER

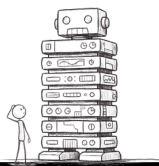
EVALUATION BEFORE

3 layers.

1. **Unit evals:** test every tool call and every prompt with known inputs and expected outputs.
2. **Integration evals:** test the full pipeline end to end, does the hospital Reception agent hand off correctly to the Physician?
3. **Adversarial evals:** can someone inject a malicious prompt and hijack your agent's behavior?

Tools:

- DeepEval for unit testing
- Ragas if you have RAG
- Promptfoo for CI/CD integration



HALLUCINATION MITIGATIONS DURING



Because LLMs hallucinate and AI agents compound hallucinations across reasoning, tools, and memory simultaneously.

3 Must Do

One: Chain-of-Verification.

Your agent generates an answer, then generates verification questions about that answer, then revises. 28% hallucination reduction. Not from a better model. From making the agent interrogate itself. In LangGraph two extra nodes.

Two: Enforce JSON.

Parse failure IS a hallucination. If your agent returns free text when you expected structured output — that's not a formatting issue, that's a reliability failure. Pydantic at the output layer is your first mitigation. You already built it in Step 2.

Three: Human-in-the-Loop for high-stakes actions.

Any action that writes, deletes, spends, or sends — requires human approval above a defined threshold. Not because the AI isn't good enough. Because the cost of being wrong once exceeds the cost of an approval step forever. That €200K order? No human checkpoint before the POST request. Never again."

MONITORING AFTER

3 signals.

1. Output schema compliance.

If your Pydantic validation pass rate drops below 95%, something upstream broke. The agent didn't crash. The data did.

2. Tool call success rate.

High retries are not a performance issue. They're a symptom. A tool is silently failing and your agent is covering for it.

3. Latency per agent node.

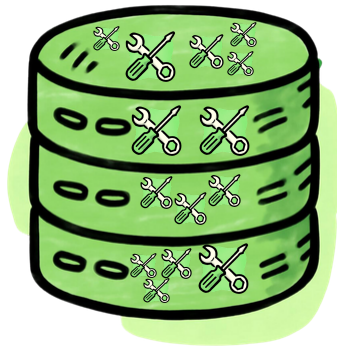
A sudden spike doesn't mean your whole system is slow, it tells you exactly which node broke, before your user notices.

Tools

- LangSmith if you're on LangChain.
- Langfuse if you want open source and self-hosted.

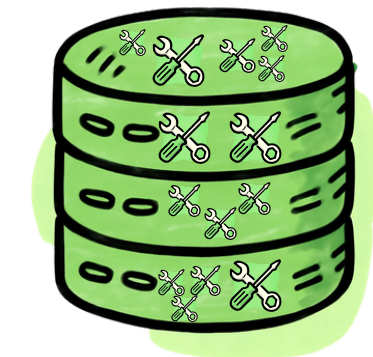


RAG FOR TOOLS



Step 1: Pre-process (done once)

- Create a registry of all your tools
- For each tool, write a description
- Convert descriptions to embeddings (vector representations)
- Store in a searchable database



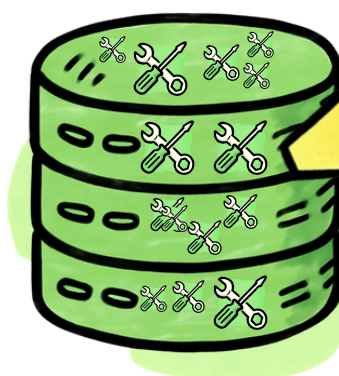
Step 2: Runtime (for each query)

User asks: "What's the weather in Tokyo?"

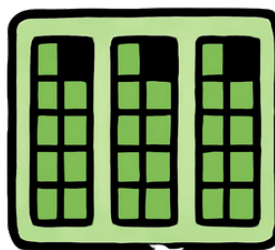


Step 3: Agent acts

Finds 5 most relevant tools:

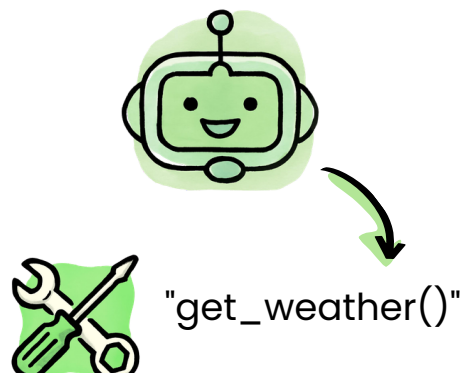
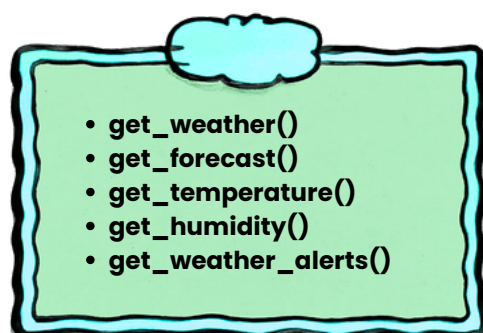


- get_weather()
- get_forecast()
- get_temperature()
- get_humidity()
- get_weather_alerts()



→ Agent ONLY sees these 5 tools (not all 100)

RAG filter Relevant Tools for Agent



17 ALGORITHMS FOR AGENTIC REASONING & PLANNING FOR PRODUCTION AI AGENTS

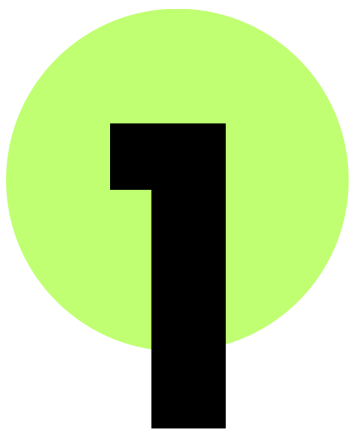


Dr. Maryam Miradi
PhD AI Scientist

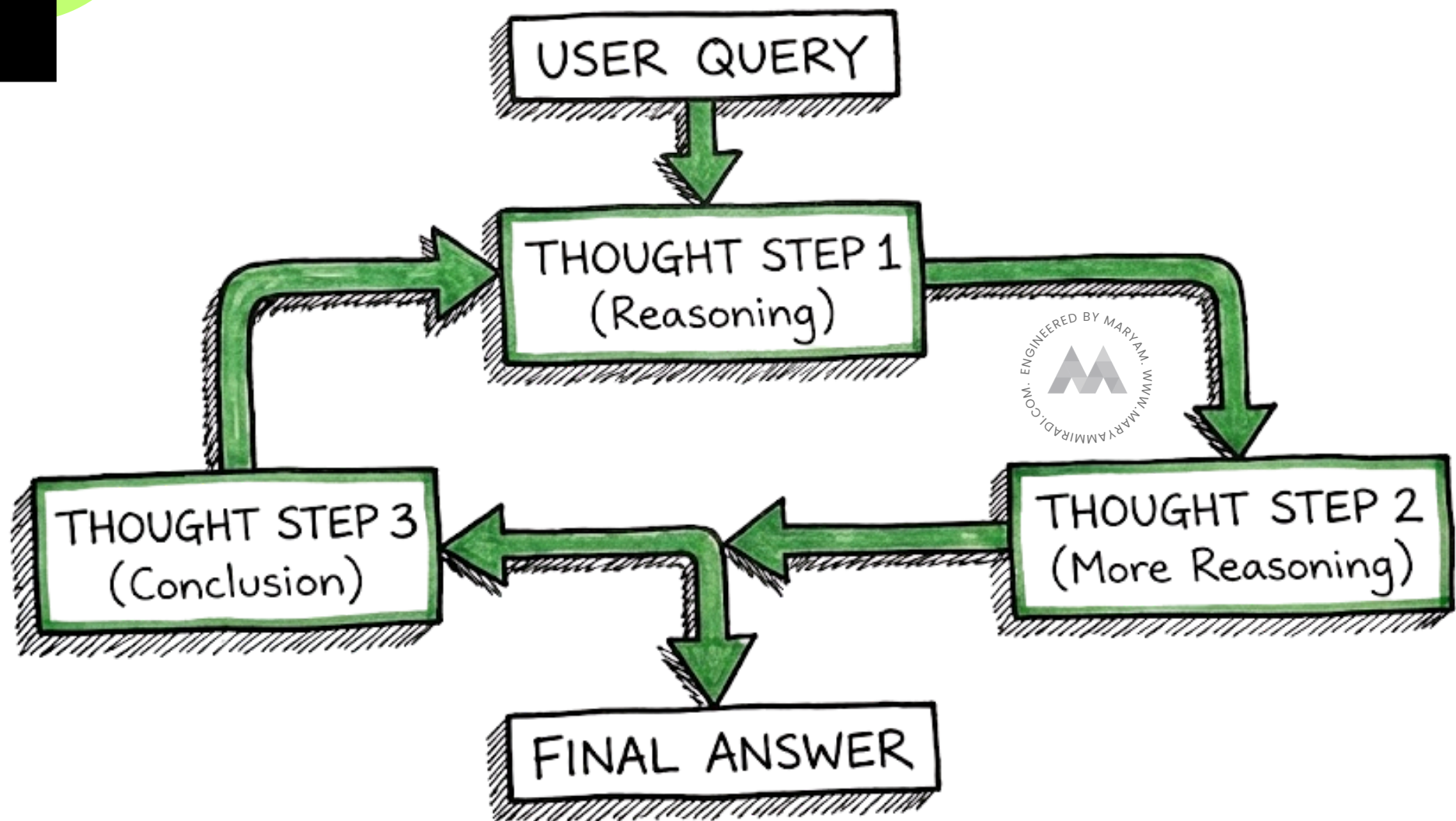


Pure Reasoning

(Internal Monologue)

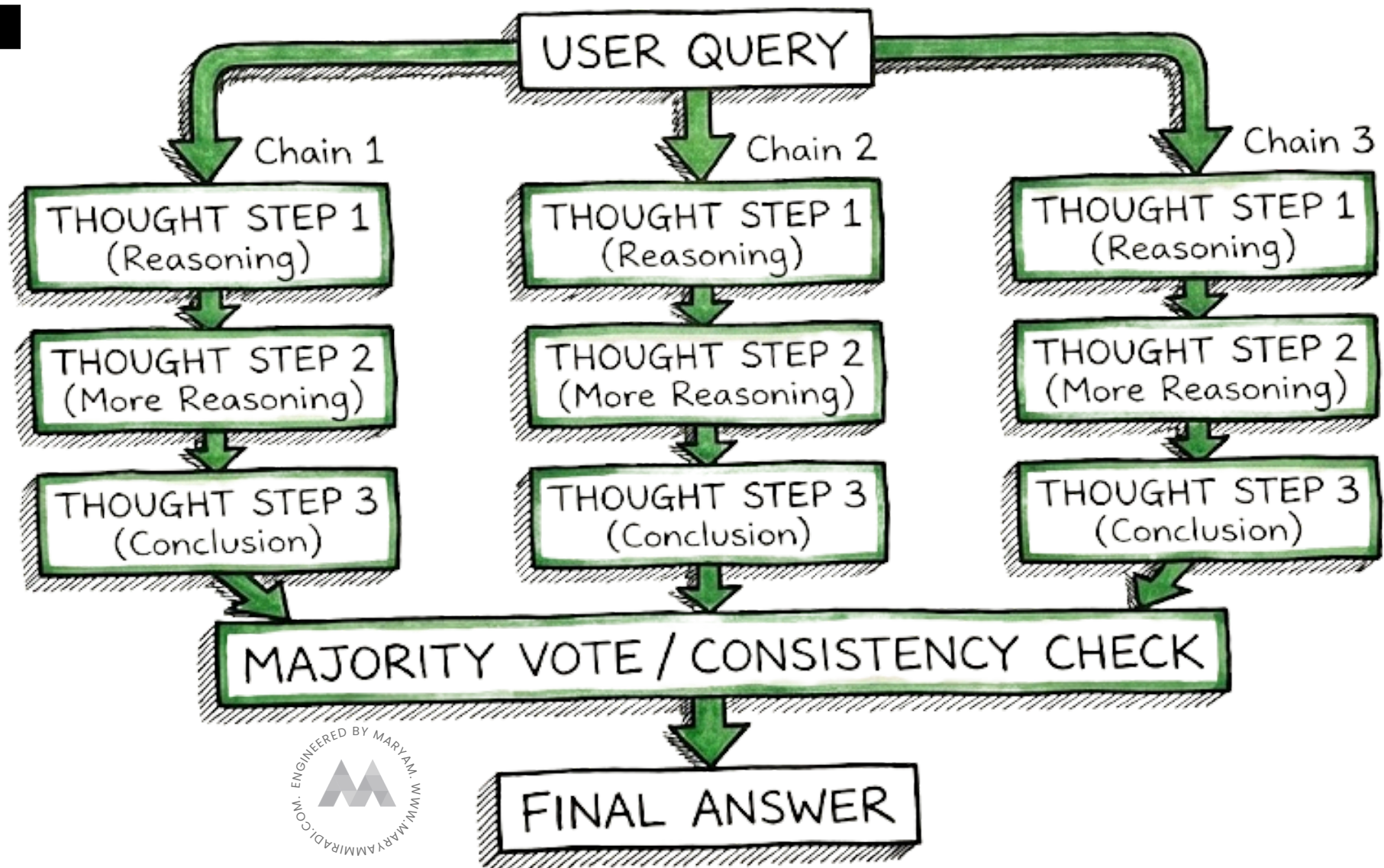


Chain-of-Thought (CoT)



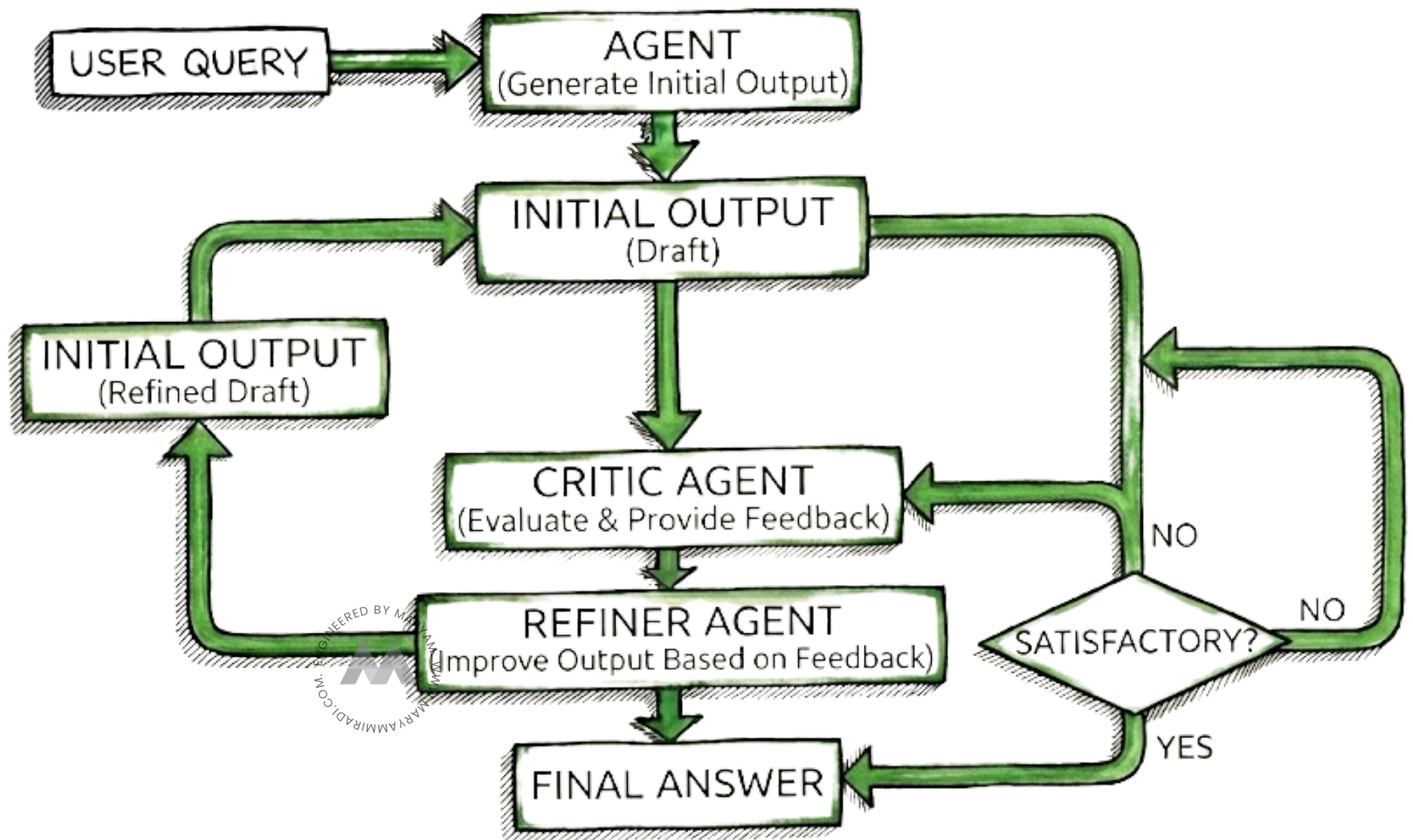
Aspect	Details
Visual Pattern	Step 1 → Step 2 → Step 3 → Answer
How It Works	Generates step-by-step reasoning before final answer, following linear logic.
Complexity	●○○○○
Best For	Math problems, logical reasoning, tasks with clear sequential steps.
Examples	Math tutors, problem-solving agents, educational tools.
Pro	Improves reasoning on complex problems, easy to implement, transparent thinking
Con	Linear thinking only, no backtracking, prone to compounding errors

Self-Consistency CoT



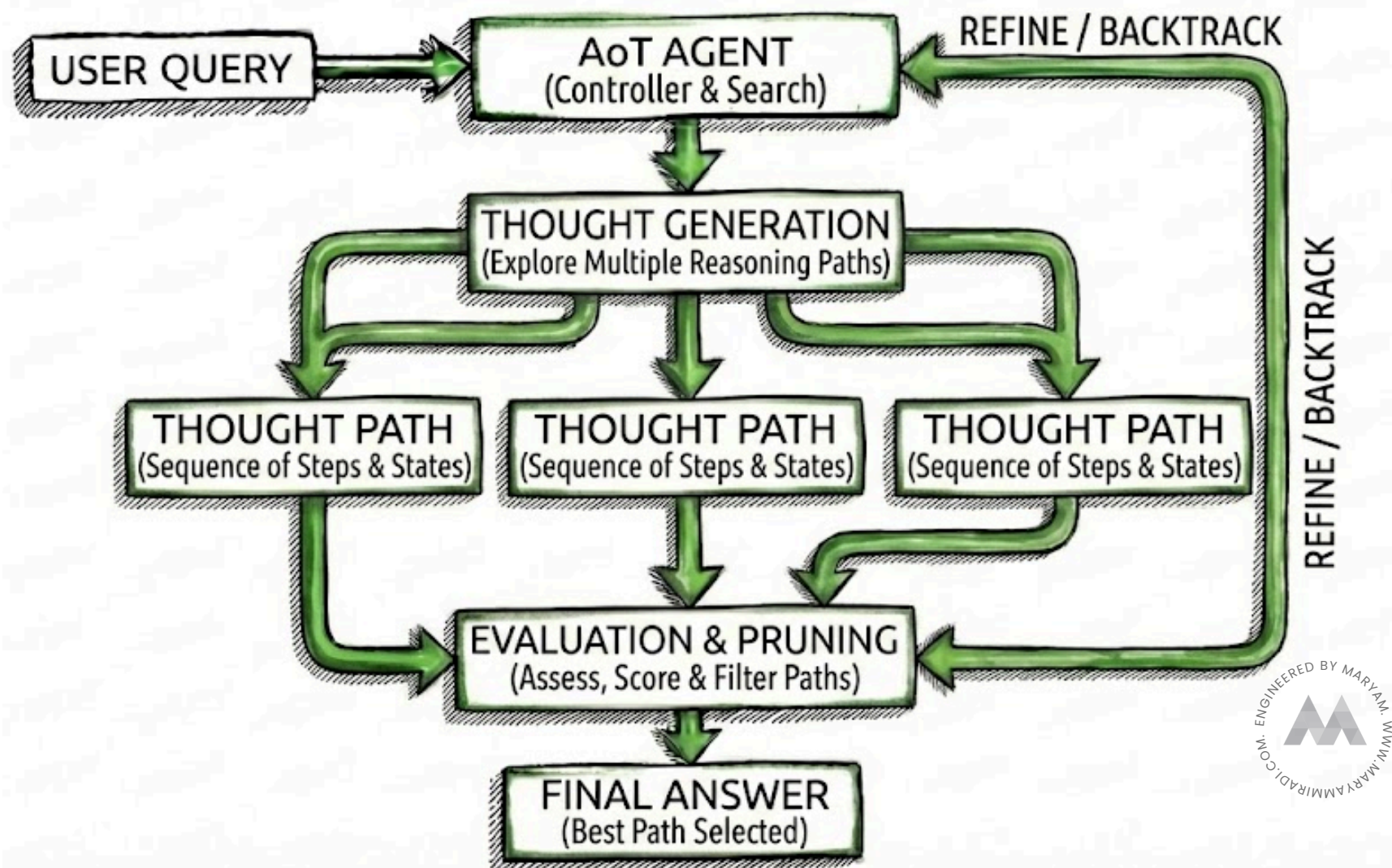
Aspect	Details
Visual Pattern	Multiple CoT paths → Voting → Final answer
How It Works	Samples multiple CoT paths, selects most consistent answer through voting/majority.
Complexity	●●○○○
Best For	Reducing hallucinations, improving accuracy, ensemble benefits.
Examples	High-stakes reasoning, fact verification, critical decisions.
Pro	More robust than single CoT, reduces hallucinations through ensemble
Con	Expensive (multiple LLM calls), no interaction with environment

Self-Refine Agent Planning



Aspect	Details
Visual Pattern	Generate → Critique → Revise → Repeat
How It Works	Iteratively criticizes and improves own output through self-evaluation cycles.
Complexity	●●○○○
Best For	Quality improvement without external feedback, writing and code tasks.
Examples	Content refinement, code improvement, answer polishing.
Pro	Simple implementation, improves output quality, works across many tasks
Con	Not grounded in external feedback, improvements may plateau

Algorithm of Thoughts (AoT) Agent Planning

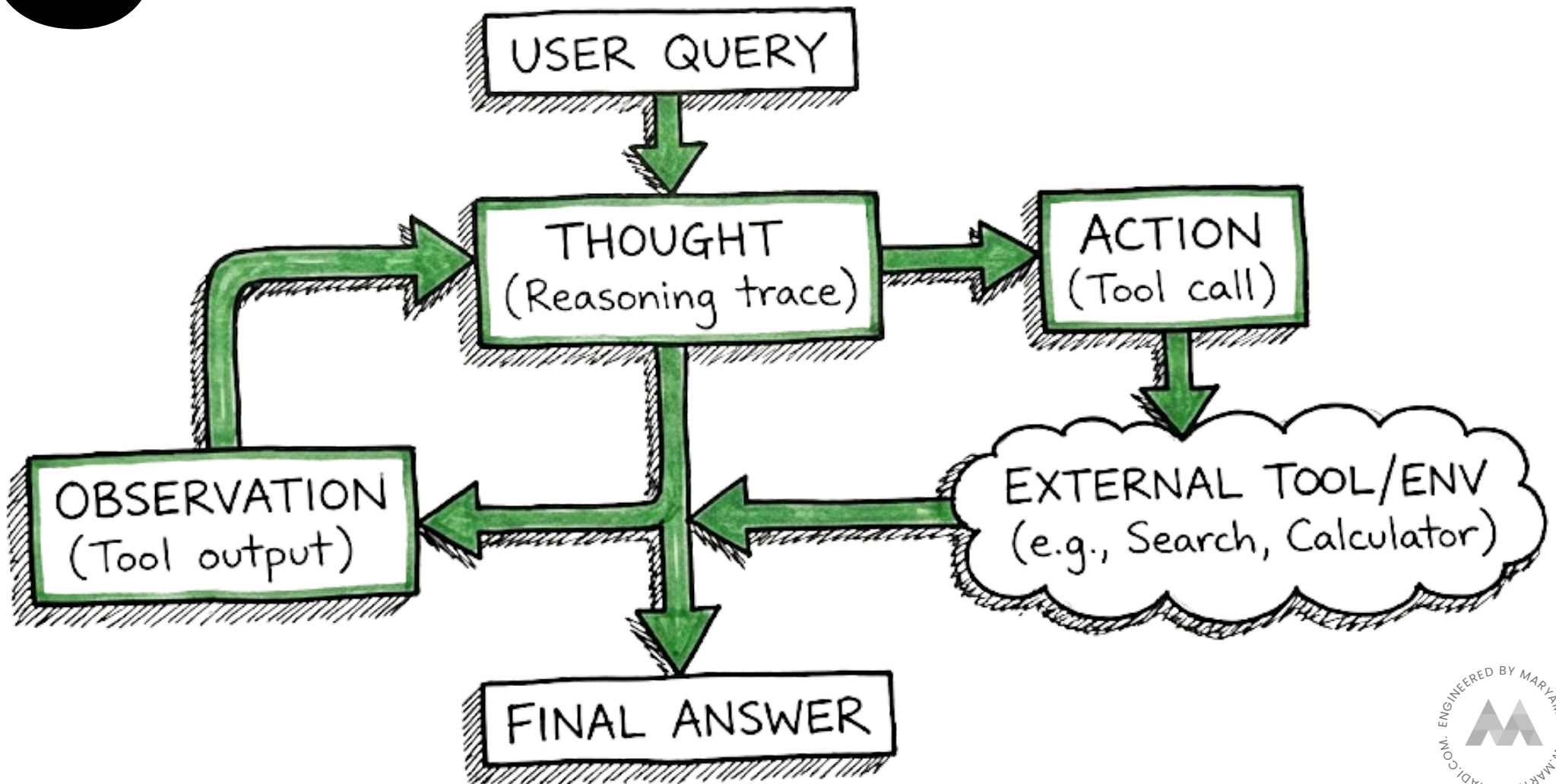


Aspect	Details
Visual Pattern	In-context search with algorithmic exploration
How It Works	Implements search algorithms within context to explore solution space efficiently.
Complexity	●●●○○
Best For	Efficient exploration, algorithmic problem-solving, structured search.
Examples	Optimization problems, algorithmic tasks, structured reasoning.
Pro	Efficient exploration, human-like problem-solving approach
Con	Requires careful prompt engineering, domain-specific design

Reasoning + Acting

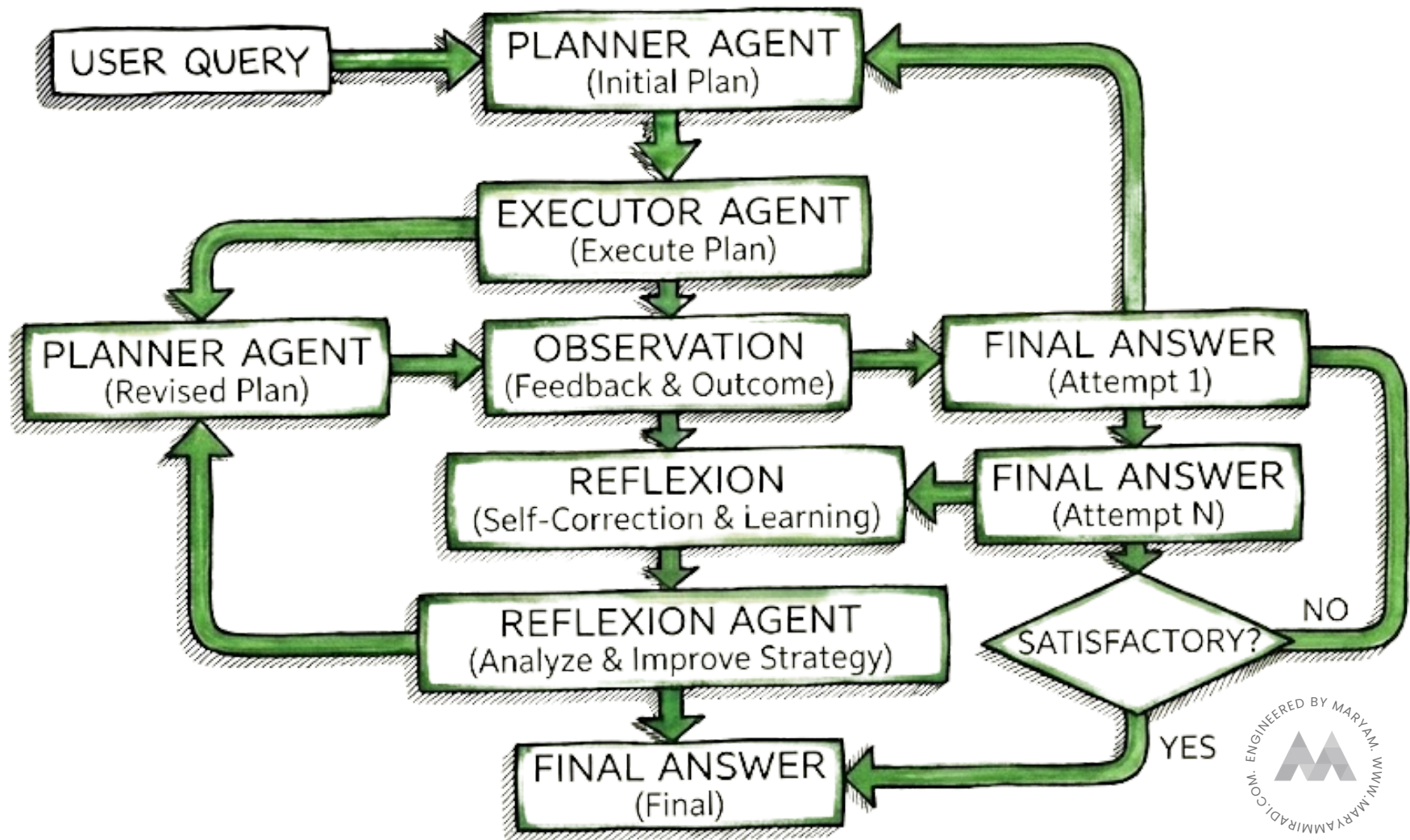
(Reactive Planning)

ReAct: Reasoning + Acting Loop



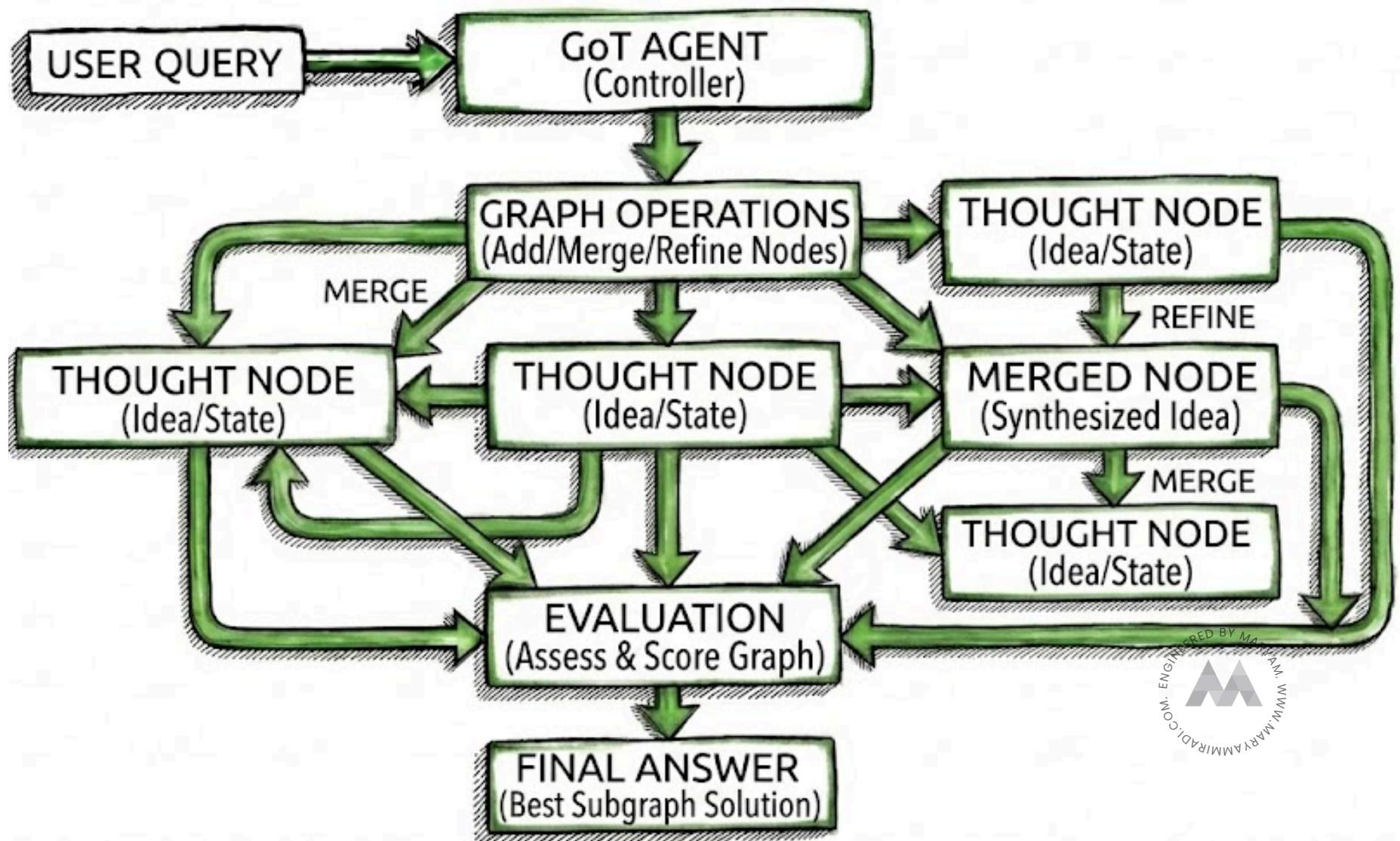
Aspect	Details
Visual Pattern	Thought→Action→Observation cycle
How It Works	Alternates between reasoning traces and actions in iterative loops with environment feedback.
Complexity	●○○○○
Best For	Simple tasks needing transparency, quick prototyping, interpretable agents.
Examples	Customer support bots, basic research assistants, simple automation.
Pro	Simple to implement, interpretable thought process, works well for straightforward tasks
Con	No lookahead planning, can get stuck in local optima, requires LLM call for each step

Reflexion Agent Planning



Aspect	Details
Visual Pattern	Try → Fail → Reflect → Try again (improved)
How It Works	Uses verbal self-reflection on failures to generate improvements stored in episodic memory.
Complexity	●●●○○
Best For	Learning from mistakes essential, iterative improvement, code generation.
Examples	Programming agents (HumanEval), sequential decision-making (AlfWorld).
Pro	Learns from failures, episodic memory for improvement, no weight updates needed
Con	Requires multiple episodes, depends on quality of self-evaluation

Graph of Thoughts (GoT) Agent Planning

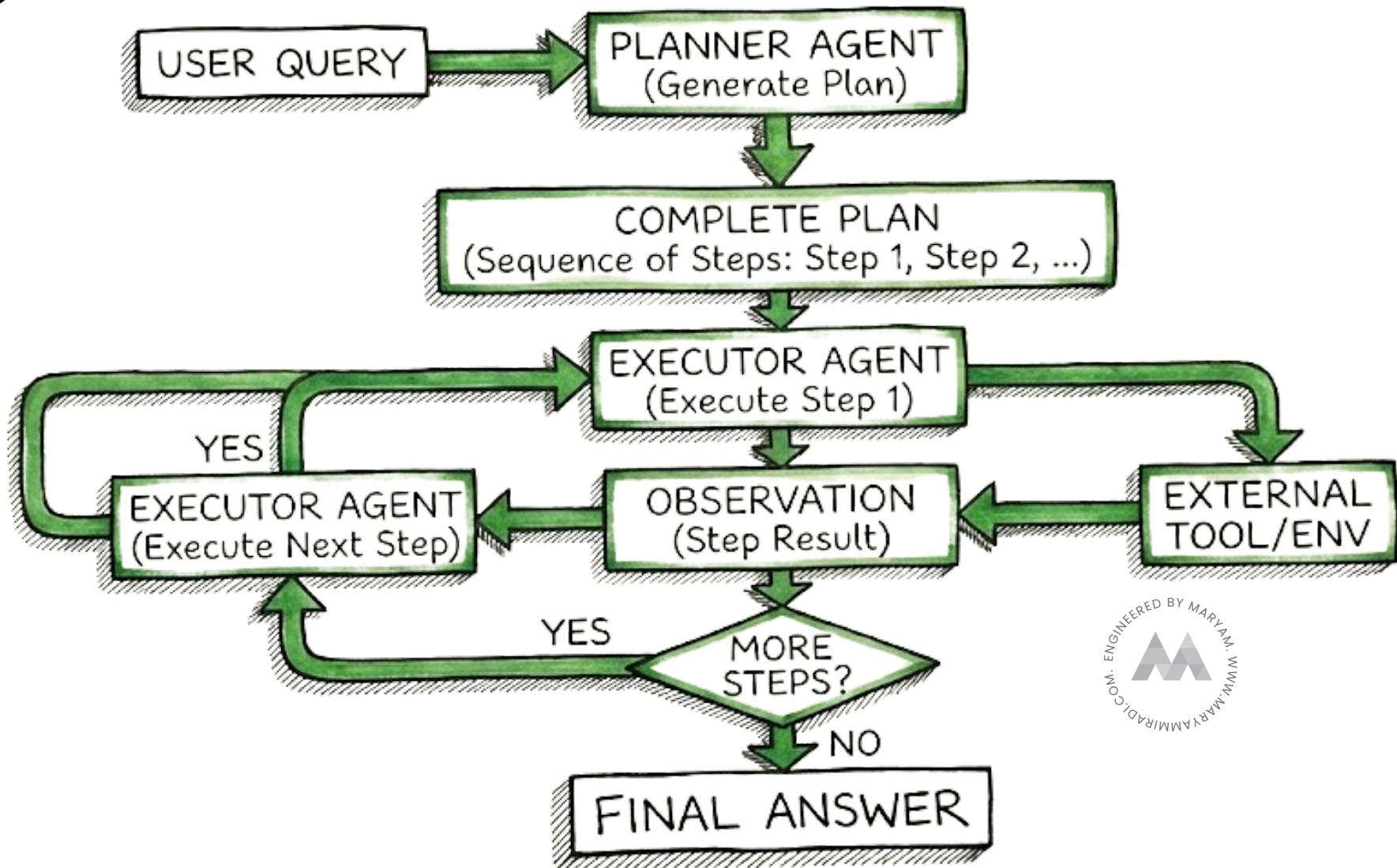


Aspect	Details
Visual Pattern	Network of interconnected thoughts with arbitrary connections
How It Works	Represents reasoning as graph with nodes (thoughts) and edges (dependencies).
Complexity	●●●●○
Best For	Complex dependencies, non-linear reasoning, interconnected ideas.
Examples	Complex analysis, multi-faceted reasoning, dependency modeling.
Pro	More flexible than tree structure, handles complex dependencies
Con	Complex to implement, harder to visualize, expensive to execute

Deliberative Planning

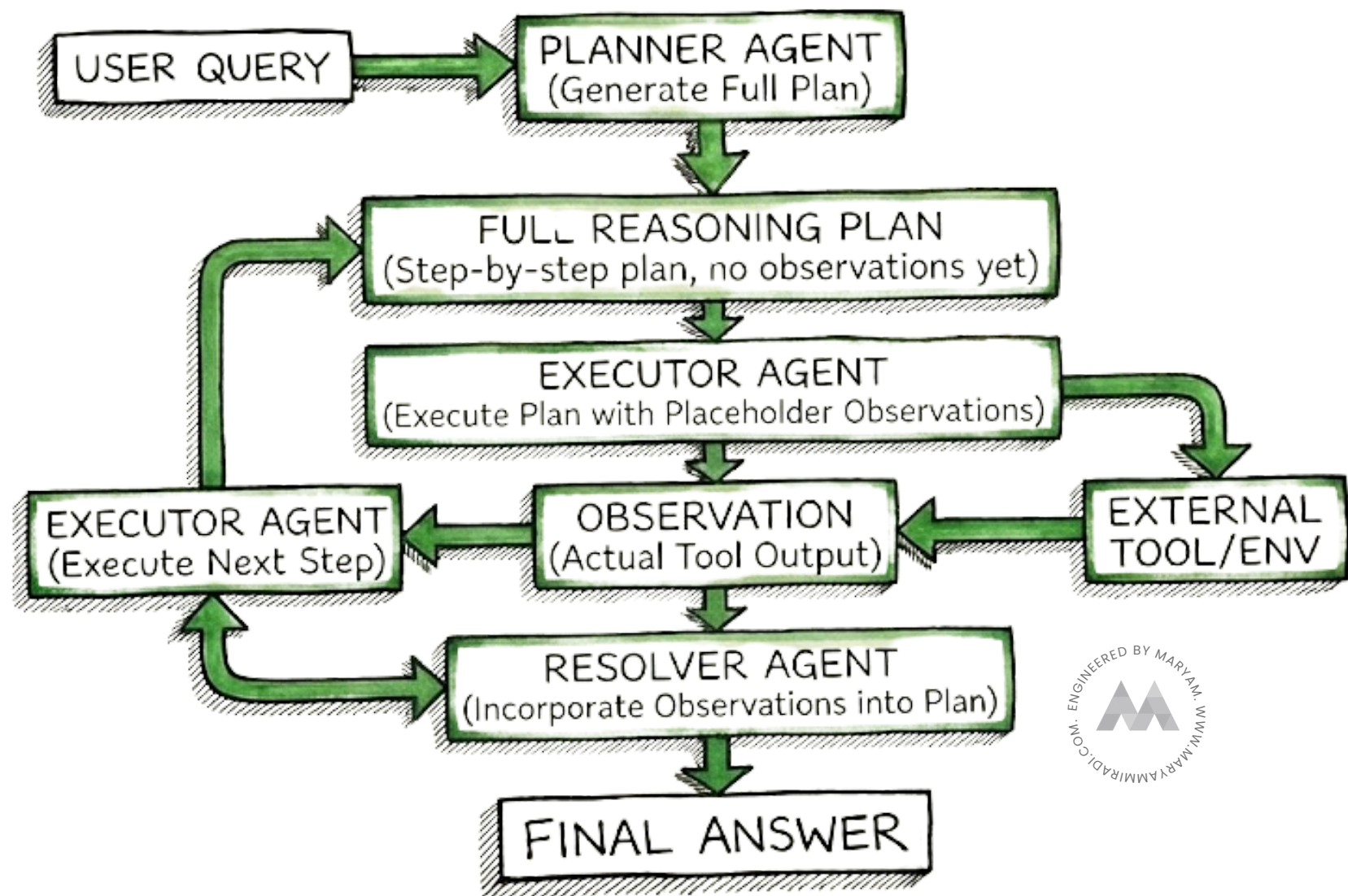
(Upfront or Strategic)

Plan-and-Execute Agent



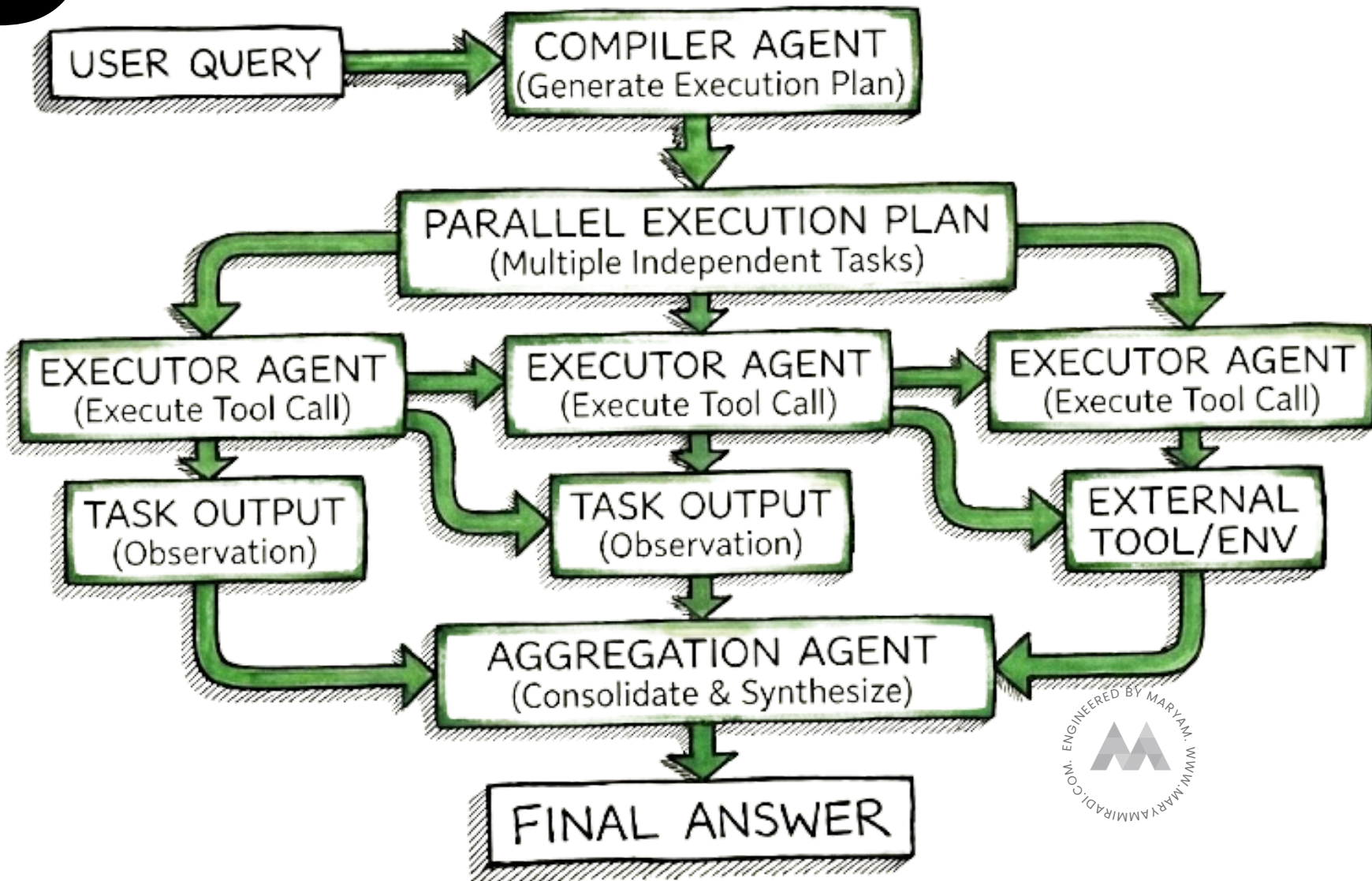
Aspect	Details
Visual Pattern	Planner → [Step 1, Step 2, Step 3] → Executor
How It Works	Creates complete plan upfront, then executes steps sequentially with minimal replanning.
Complexity	●●○○○
Best For	Multi-step workflows, cost optimization important, predictable tasks.
Examples	Travel planning, data processing pipelines, report generation.
Pro	Cost-effective (fewer LLM calls), explicit global planning, better for multi-step tasks
Con	Plan can become outdated, limited adaptability during execution

ReWOO (Reasoning Without Observation)



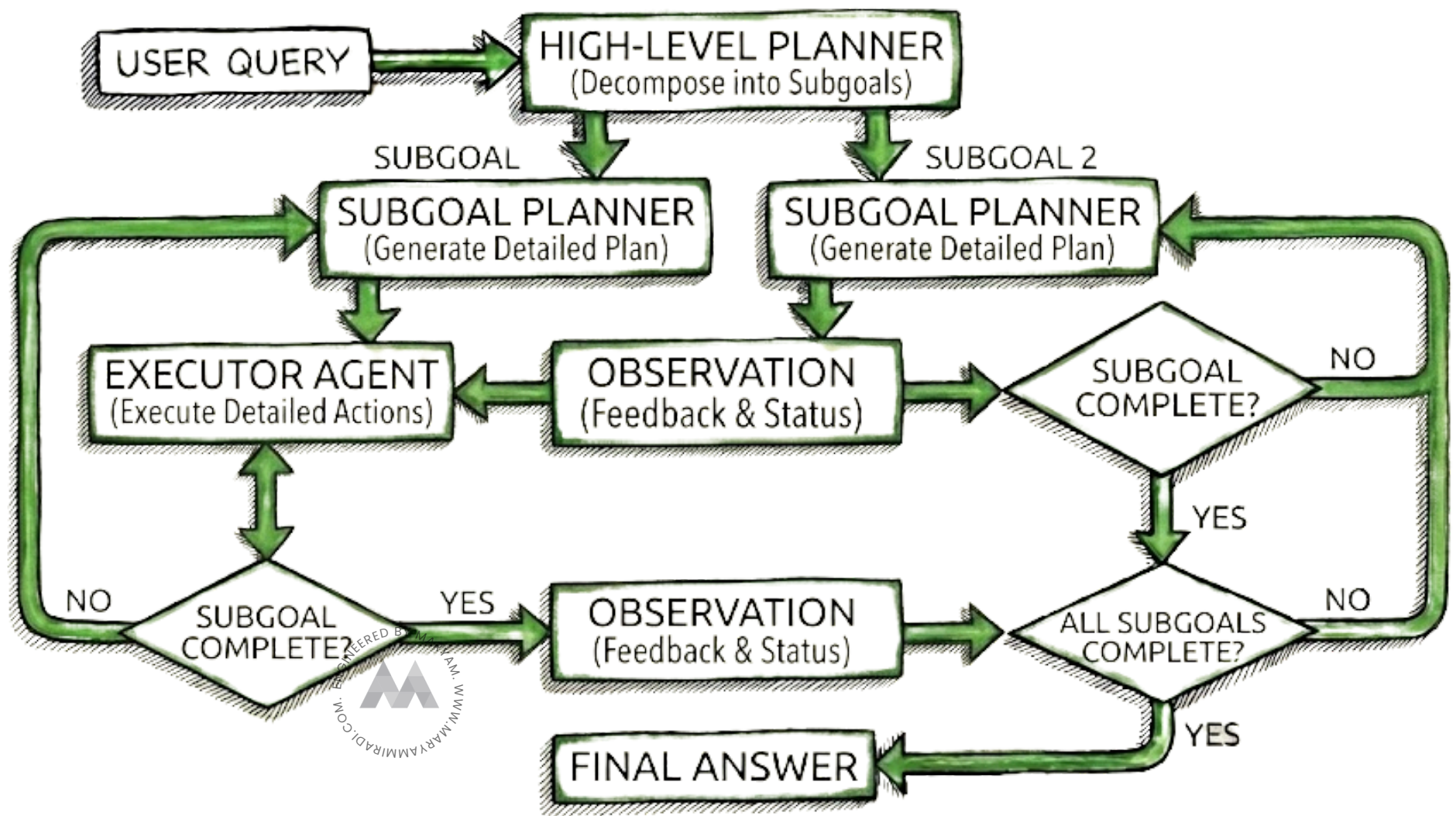
Aspect	Details
Visual Pattern	Plan: #E1, #E2, #E3 → Execute referencing variables
How It Works	Plans with variable assignment (#E1, #E2), executes without LLM per task, references prior outputs.
Complexity	●●●○○
Best For	Efficient execution needed, tasks with dependencies, reduce LLM calls.
Examples	Multi-step research, data transformation workflows.
Pro	No LLM needed for each task, can reference previous outputs, more efficient
Con	Still sequential execution, requires accurate upfront planning

LLMCompiler Agent Planning



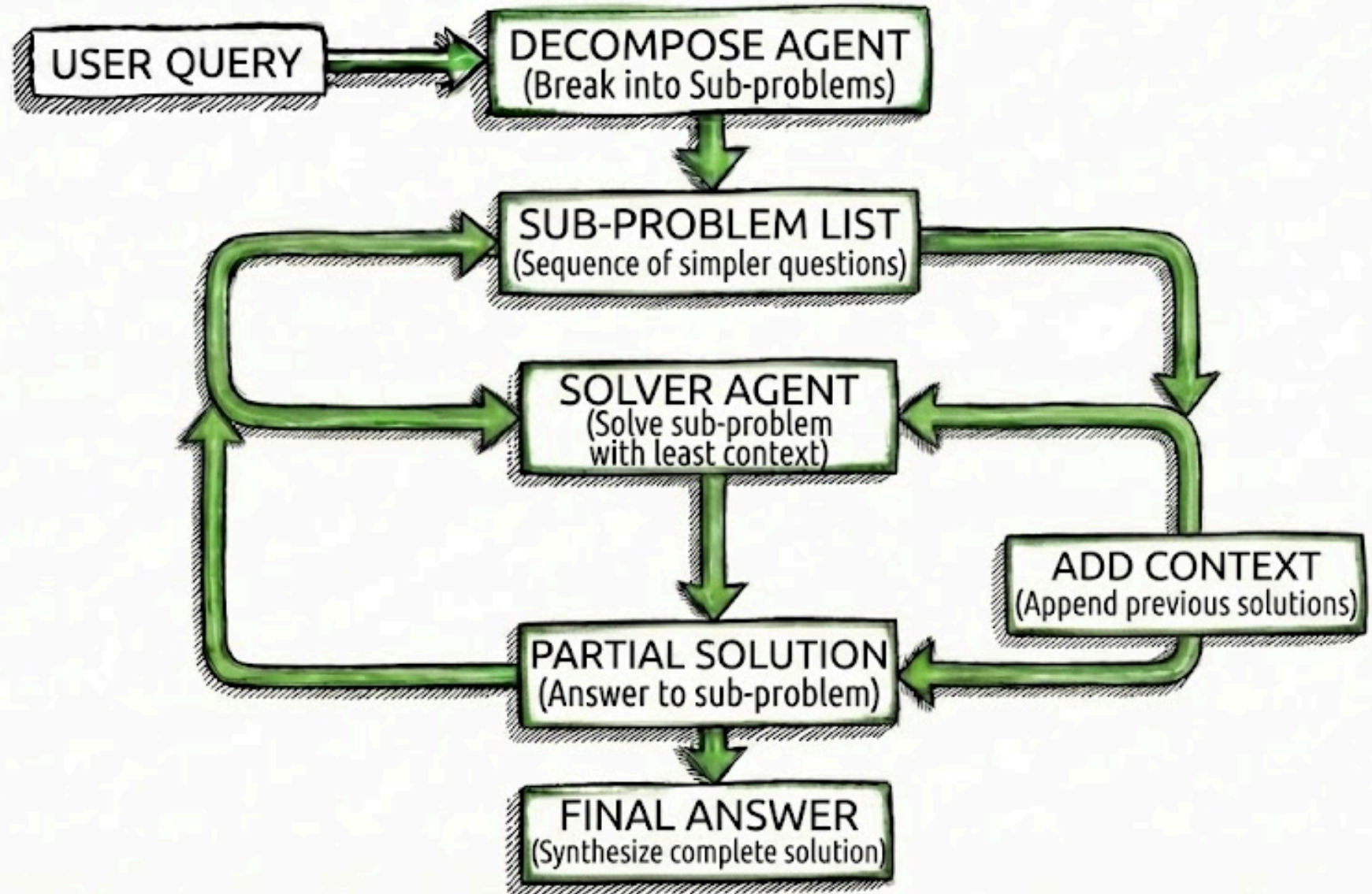
Aspect	Details
Visual Pattern	Task DAG with parallel execution paths
How It Works	Creates DAG of tasks, executes independent tasks in parallel for maximum speed.
Complexity	●●●●○
Best For	Speed critical, independent parallel tasks, low-latency requirements.
Examples	Real-time analysis, parallel data processing, fast API orchestration.
Pro	Fastest execution through parallelization, reduces latency, efficient orchestration
Con	Complex dependency management, requires task independence analysis

Hierarchical Planning Agent



Aspect	Details
Visual Pattern	High-level goal → Mid-level subgoals → Low-level actions
How It Works	Decomposes high-level goals into subgoals and primitive actions at multiple levels.
Complexity	●●●○○
Best For	Long-horizon tasks, complex goal structures, structured decomposition.
Examples	Robot navigation, enterprise workflows, multi-stage projects.
Pro	Handles complexity through decomposition, clear goal hierarchy
Con	Rigid structure, complex to implement, requires careful goal design

Least-to-Most Prompting Agent Planning

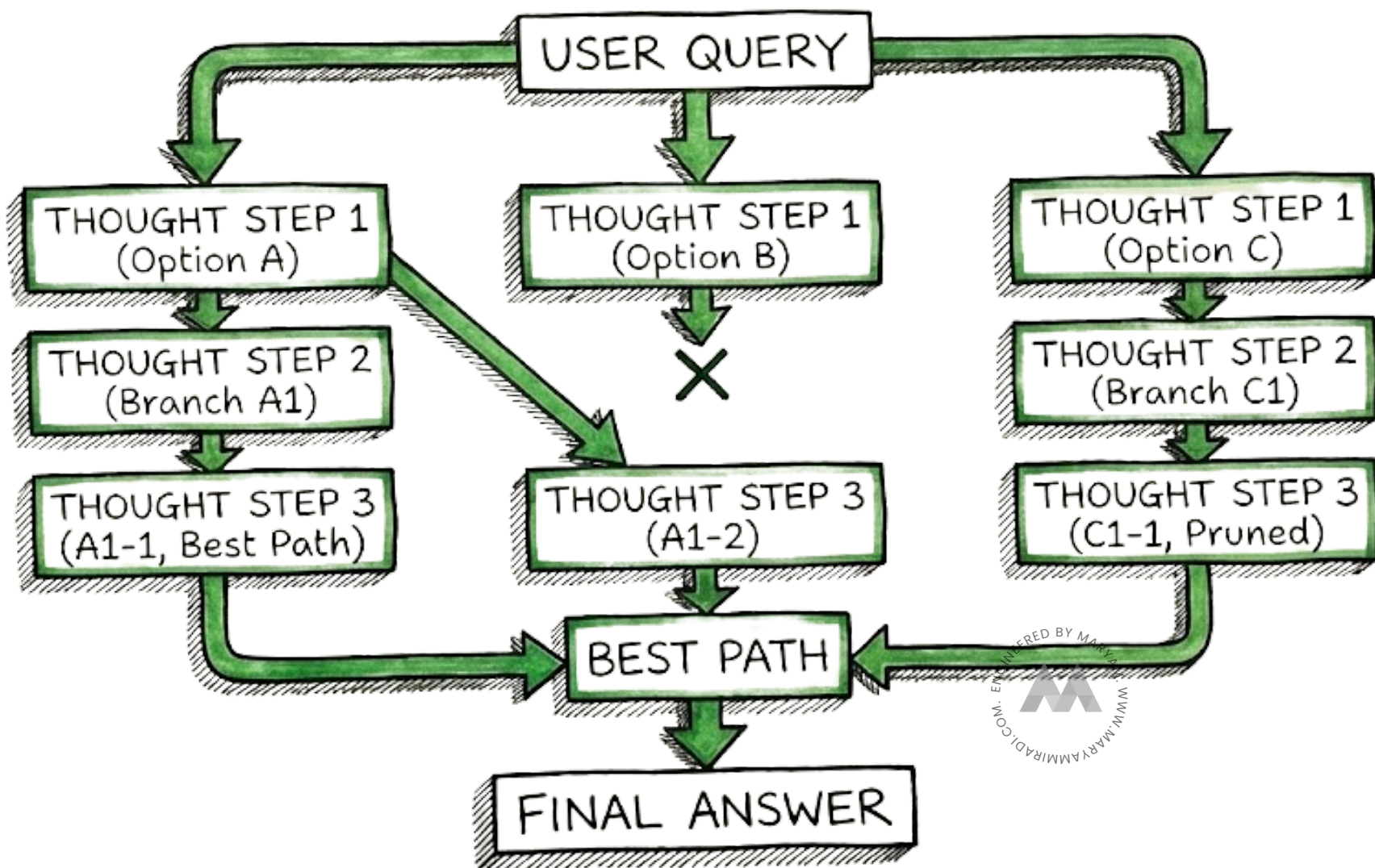


Aspect	Details
Visual Pattern	Complex problem → Simpler subproblems → Solve easiest first
How It Works	Breaks problems into progressively simpler subproblems, solves smallest first.
Complexity	●●○○○
Best For	Compositional reasoning, hierarchical problems, educational contexts.
Examples	Math problem solving, compositional tasks, step-by-step learning.
Pro	Systematic problem breakdown, good for compositional reasoning
Con	Linear decomposition may miss optimal paths, not suitable for all tasks

Advanced Search & Optimization

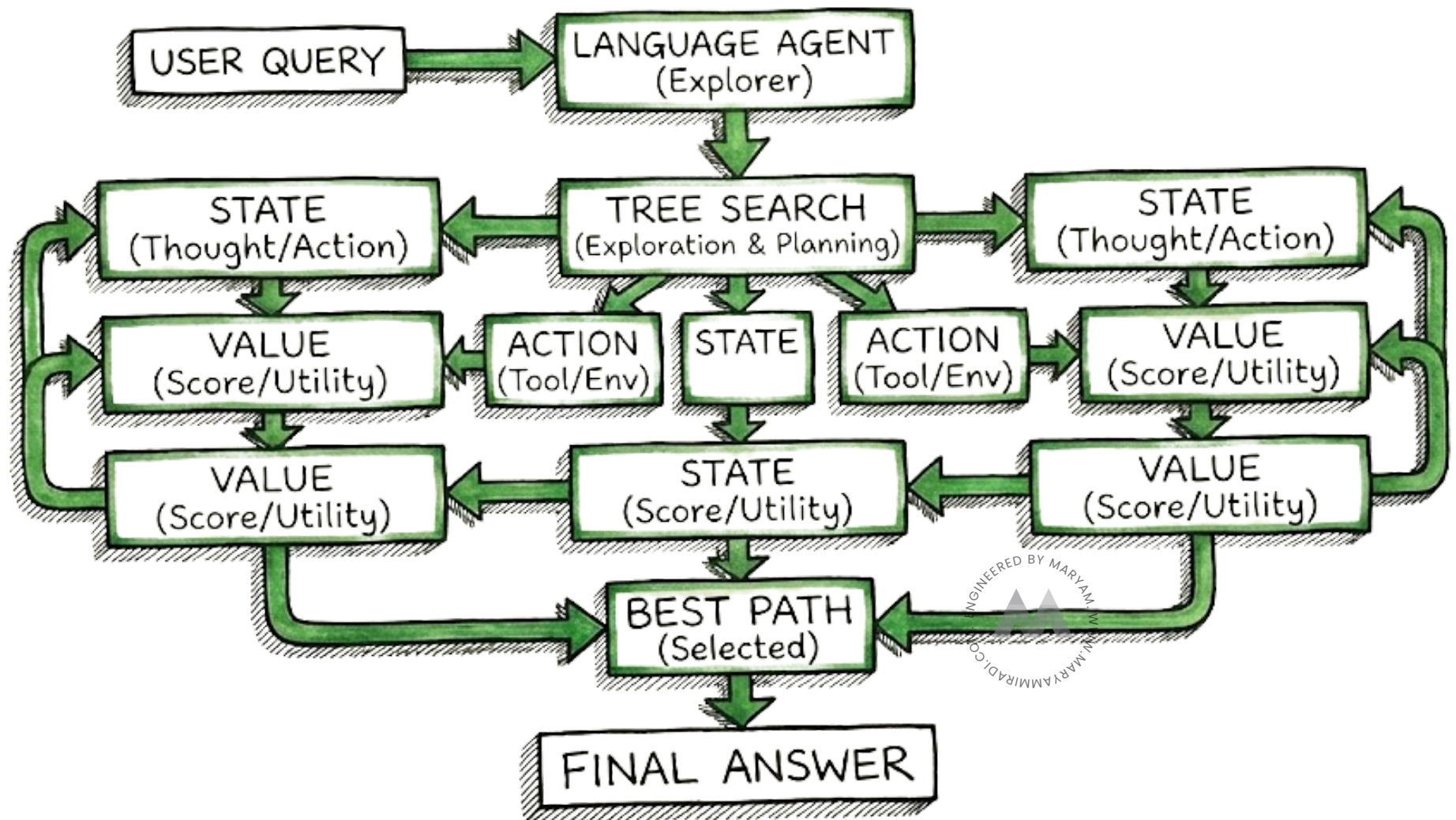
(The "Agentic" Gold Standard)

Tree of Thoughts (ToT)



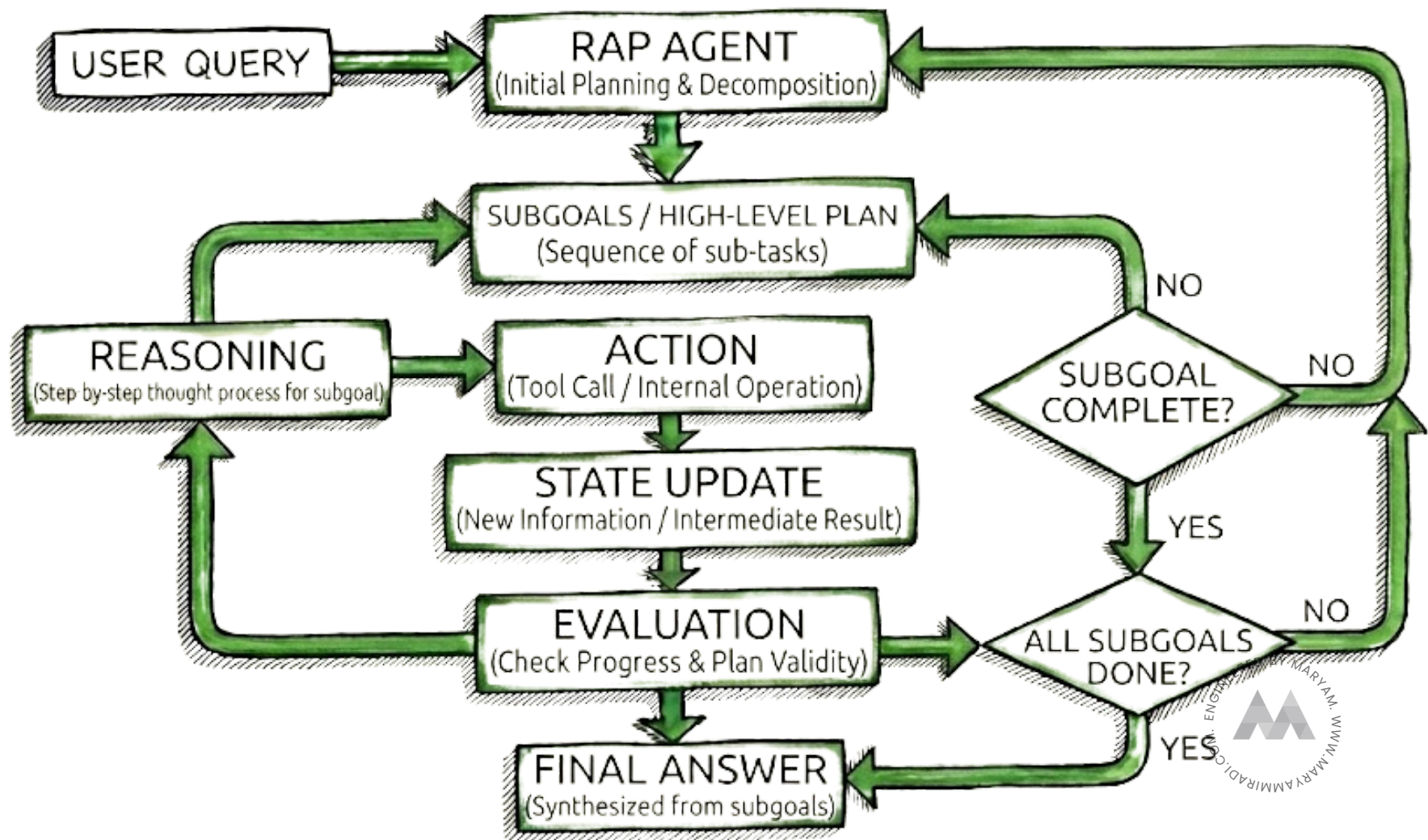
Aspect	Details
Visual Pattern	Root → Multiple branches → Evaluate → Backtrack
How It Works	Explores multiple reasoning paths using BFS/DFS, evaluates and backtracks as needed.
Complexity	●●●○○
Best For	Complex problems requiring exploration, strategic lookahead critical.
Examples	Game playing (24 game), creative writing, puzzle solving.
Pro	Explores multiple reasoning paths, enables backtracking, deliberate problem-solving
Con	Computationally expensive, can explore low-value paths unnecessarily

Language Agent Tree Search (LATS)



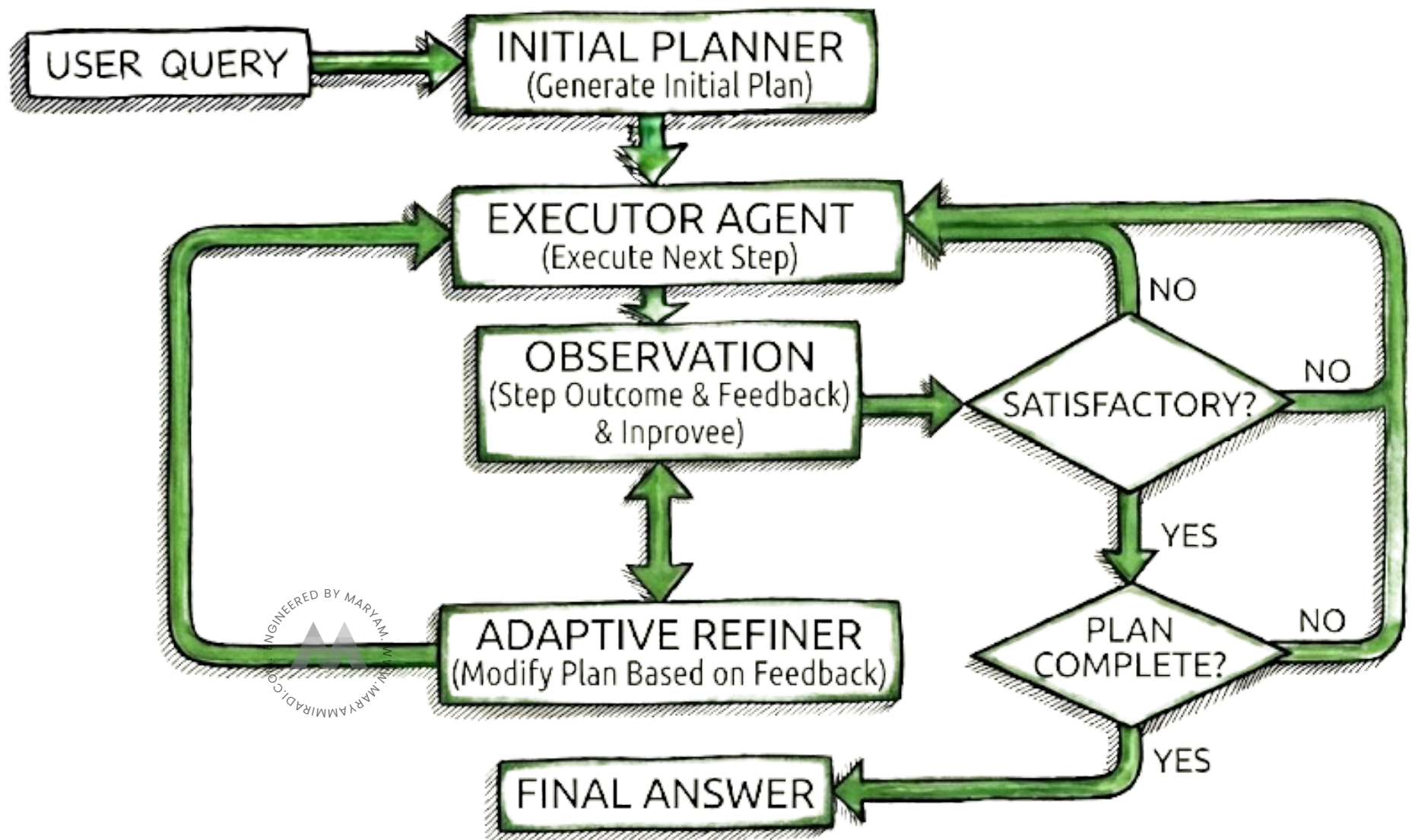
Aspect	Details
Visual Pattern	MCTS tree with selection → expansion → simulation → backpropagation
How It Works	Combines MCTS with LLM-powered value functions and self-reflections for optimal exploration.
Complexity	●●●●●
Best For	Highest quality needed, complex reasoning+acting, external feedback available.
Examples	Advanced coding (92.7% HumanEval), web navigation, interactive QA.
Pro	Unifies reasoning, acting, planning; superior performance; balances exploration-exploitation
Con	Very high computational cost, complex implementation, sensitive to reward quality

RAP (Reasoning via Planning) Agent Planning



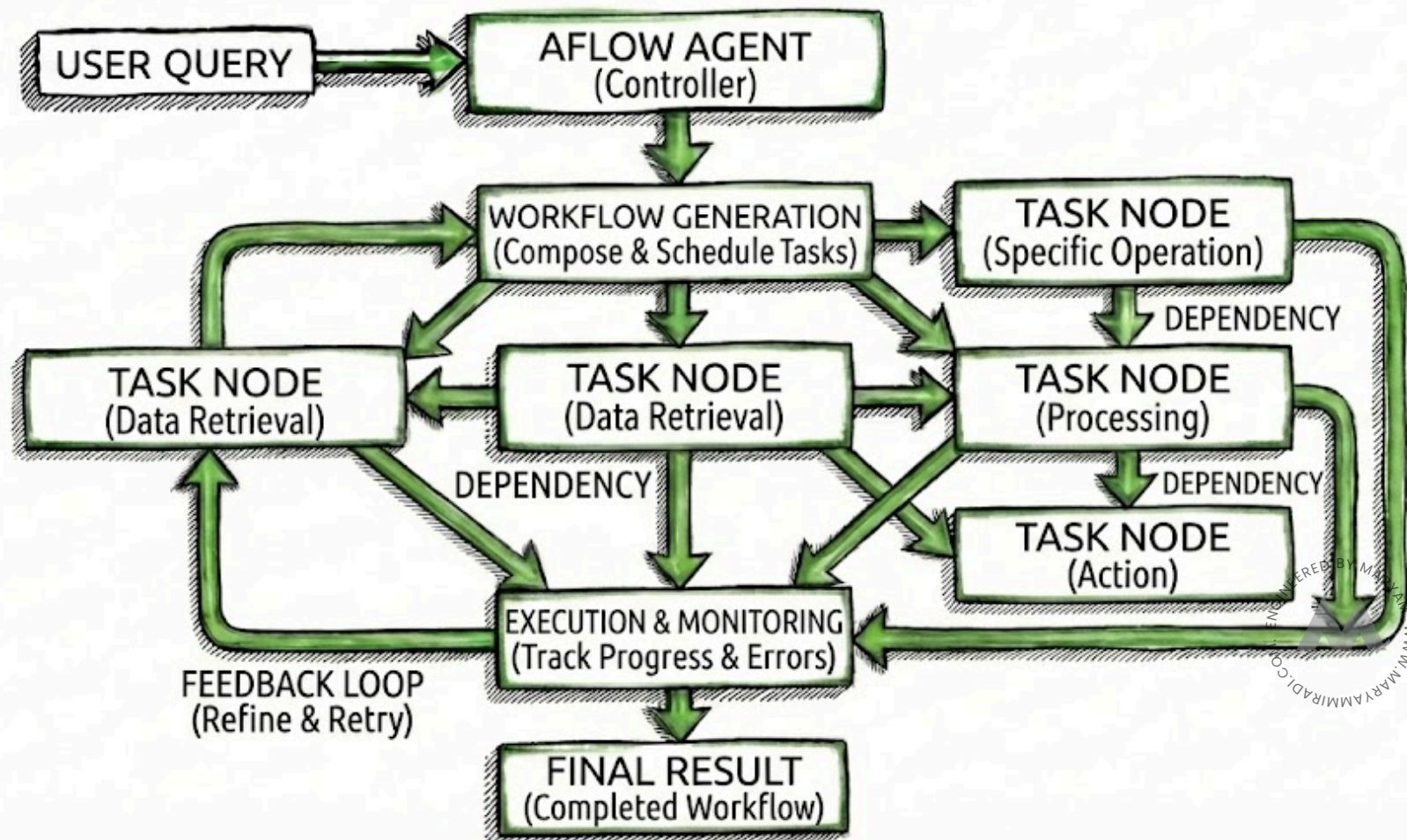
Aspect	Details
Visual Pattern	MCTS with LLM world model simulations
How It Works	Uses MCTS with LLM as world model to simulate future states and plan strategically.
Complexity	●●●●○
Best For	Strategic planning needed, internal simulation sufficient, no external environment.
Examples	Strategic reasoning, hypothetical planning, constrained environments.
Pro	Strategic exploration, uses LLM as world model, principled planning algorithm
Con	Relies only on internal LLM knowledge, cannot adapt to external feedback

Adaptive Planning (AdaPlanner) Agent Planning



Aspect	Details
Visual Pattern	Plan → Execute → Feedback → Replan loop
How It Works	Refines plans dynamically based on environmental feedback and execution results.
Complexity	●●●○○
Best For	Dynamic environments, plan adaptation critical, unpredictable obstacles.
Examples	Robotics, interactive agents, changing requirements.
Pro	Adapts to environment response, handles dynamic situations well
Con	Overhead from re-planning, can be slow for simple tasks

AFLOW (Automated Workflow) Agent Planning



Aspect	Details
Visual Pattern	MCTS over workflow operators and configurations
How It Works	Uses MCTS to automatically discover optimal agent workflows and patterns.
Complexity	●●●●●
Best For	Workflow optimization, discovering best agent patterns, research settings.
Examples	Agent architecture search, workflow discovery, research experiments.
Pro	Automatically optimizes workflows, discovers novel patterns
Con	Extremely computationally expensive, requires extensive search